

Quiz #3

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, March 26

Name	Hanbin Zheng
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 8 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1 _____/10

Problem 2 _____/10

Problem 3 _____/10

Problem 4 _____/10

Total _____/40



Problem 1 (10 points)**1. Multiple-choice question**

Select ALL options that satisfy the requirements of the question and mark them in the corresponding area on the answer sheet.

Choosing any incorrect option earns zero points for the question; selecting only some of the correct options earns one point; selecting all correct options earns two points.

(d) (1) What is the worst-case asymptotic time complexity of inserting a new element at a given position in an array-based list of length n ?

- a) $O(1)$
 b) $O(n/2)$
 c) $O(\sqrt{n})$
 d) $O(n)$
 e) $O(n^2)$



n front + move $O(n)$ - 一定是 $O(n^2)$

傻逼题目,

(c) (2) Suppose A and B are singly linked lists with lengths m and n , respectively. Only the head pointers of the two lists are given, and no tail pointers are maintained. To concatenate the two lists, list B is linked directly to the end of list A by pointer manipulation only. What is the worst-case asymptotic time complexity of this operation?

- a) $O(1)$
 b) $O(n)$
 c) $O(m)$
 d) $O(m+n)$
 e) $O(mn)$

only head

m次

同上, 傻逼题目.

(f) (3) A node in a doubly linked circular list is defined as follows:

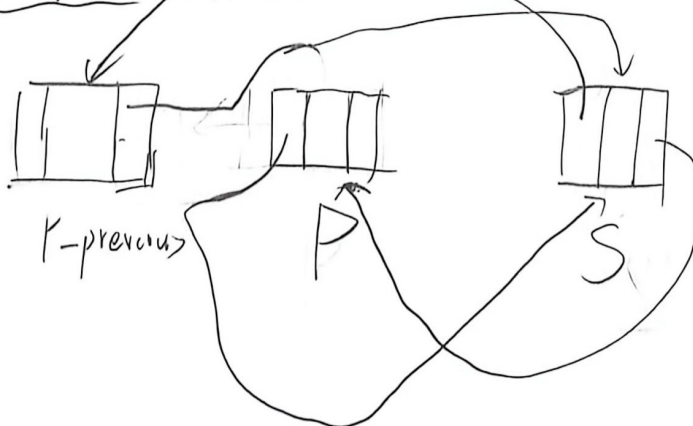
```
class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
};
```

Assume p points to a node already in the list, and s points to a newly allocated node not currently in any list. Insert s immediately before p by pointer manipulation only.

A statement sequence is considered correct if, after all statements in the sequence are executed in order, the final list structure is correct.

Which of the following statement sequences are correct?

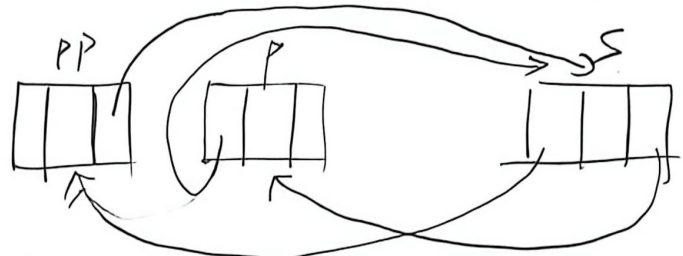
- a) $s \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $s \rightarrow \text{next} = p;$
 $p \rightarrow \text{prev} \rightarrow \text{next} = s;$
 $p \rightarrow \text{prev} = s;$



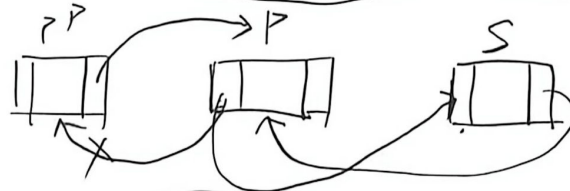
Name: _____

3

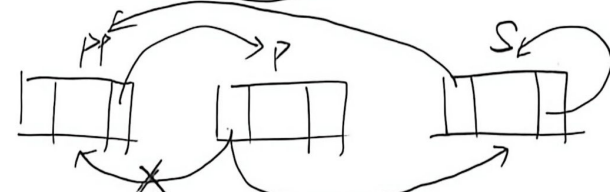
b) $p \rightarrow \text{prev} \rightarrow \text{next} = s;$
 $s \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $s \rightarrow \text{next} = p;$
 $p \rightarrow \text{prev} = s;$



c) $s \rightarrow \text{next} = p;$
 $p \rightarrow \text{prev} = s;$
 $s \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $s \rightarrow \text{prev} \rightarrow \text{next} = s;$



d) $s \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $p \rightarrow \text{prev} = s;$
 $p \rightarrow \text{prev} \rightarrow \text{next} = s;$
 $s \rightarrow \text{next} = p;$



e) $s \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $s \rightarrow \text{next} = p;$
 $p \rightarrow \text{next} \rightarrow \text{prev} = s;$
 $p \rightarrow \text{prev} = s;$



2. Fill in the blanks

Consider an unsorted ArrayList of one-character strings.

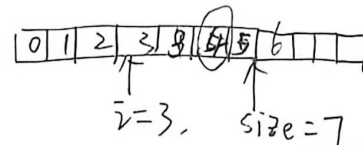
Assume the list has enough capacity, and all indices are 0-based.

The following member functions are available:

```
int f(const string& x) {
    for (int i = 0; i < size; i++) {
        if (data[i] == x) return i;
    }
    return -1;
}
```

找 index? find

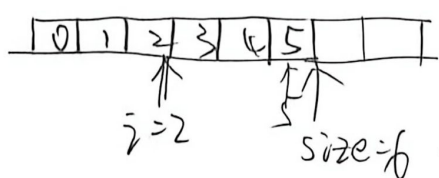
```
void g(int i, const string& x) {
    for (int k = size; k > i; k--) {
        data[k] = data[k - 1];
    }
    data[i] = x;
    size++;
}
```



插入, 在位置 i index i

所有

```
void h(int i) {
    for (int k = i; k < size - 1; k++) {
        data[k] = data[k + 1];
    }
    size--;
}
```



删除第 i 个





size = 5

i = 2, k > 3

k = 3, k = 4

k = i + 1

4 k = 3 = i + 1

Name:

```
void p(const string& x, const string& y) {
    int i = f(x);
    for (int k = size; k > i + 1; k--) {
        data[k] = data[k - 1];
    }
    data[i + 1] = y;
    size++;
}
```

把

从 i 开始
向后移动一位

在 i 处插入 "y"

Initially, Array list is ["C", "5"]

Now execute the following statements in order:

1. g(1, "S");

2. g(f("5"), "2");

3. p("2", "2");

4. h(f("5") - f("S"));

5. g(f("5") + f("S"), "2");

6. p("C", "S");

7. h(f("S"));

The content of Array list after step 3 and step 7 is:

h(1)

Problem 2 (10 points): Array List and Linked List Operations

1. Consider the following Array List code:

```
void insertAtFront(ArrayList& arr, int x) {
    if (arr.size == arr.capacity) {
        resize(arr);
    }
    for (int i = arr.size - 1; i >= 0; i--) {
        arr.data[i + 1] = arr.data[i];
    }
    arr.data[0] = x;
    arr.size++;
}
```

(a) Fill in the blank.

(b) What is the worst-case running time of this operation?

resize
(n) + O(1)

2. Suppose resize doubles the capacity each time the array is full. What is the amortized time per insertion? Briefly state the result.

3. Consider insertAtIndex for an Array List:

```
for (int i = arr.size; i >= idx; i--) {
    arr.data[i + 1] = arr.data[i];
}
```

arr.data[idx] = x;

(a) Fill in the blanks.

(b) What is the worst-case running time of this operation?



4. Consider findData(arr, x) on an Array List.

State the running time on:

unsorted array list: _____

sorted array list (using best method): _____

5. Does sorting reduce the worst-case running time of insertAfterElement in an Array List?

Answer YES or NO and briefly justify.

6. Consider the following Linked List code:

```
void insertAtFront(LinkedList& list, int x) {
```

```
    Node* n = new Node;
```

```
    n->value = x;
```

```
    n->next = list.head;
```

```
    list.head = n;
```

```
}
```

(a) Fill in the blanks.

(b) What is the running time of this operation?

7. Consider findData(list, x) in a Linked List.

State the running time on:

unsorted linked list: _____

sorted linked list: _____

8. A student claims: "Sorted linked lists allow binary search, so find is $O(\log n)$."
Is this correct? Briefly justify.



Problem 3 (10 points): Stack and Queue implementation

Consider the following partial class definitions:

```

template <class T>
class Stack {
public:
    // constructors and destructors omitted
    void enlarge();           // resize the stack to 2x size
    void push(const T& x);
    T pop();
    bool isEmpty() const;
    bool isFull() const;
private:
    T* arr_;
    unsigned size_, count_;
};

template <class T>
class Queue {
public:
    // constructors and destructors omitted
    void enlarge();           // resize the queue to 2x size
    void enqueue(const T& x);
    T dequeue();
    bool isEmpty() const;
    bool isFull() const;
private:
    T* arr_;
    unsigned size_, count_, front_, back_;
};

```

Implement the following methods:

1. Stack::push
2. Stack::pop
3. Queue::enqueue
4. Queue::dequeue
5. Stack::isEmpty

Assume:

- size_ is the current capacity of the array.
- count_ is the number of elements currently stored.
- For the stack, the top element is stored at index count - 1.
- For the queue, front_ points to the front element, and back_ points to the next available position for insertion.
- The queue is implemented as a circular array.
- If pop() or dequeue() is called on an empty structure, use exit(-1).
- You must handle the case when the structure is full before inserting.



Problem 4 (10 points): iterator & recursion**1. Considering the following code:**

```

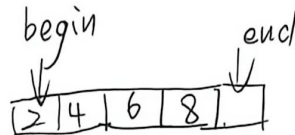
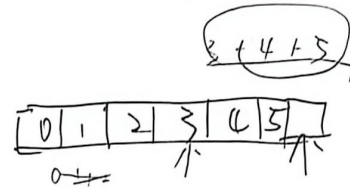
#include <iostream>
#include <vector>
using namespace std;

int mystery(vector<int>& v, vector<int>::iterator it) {
    if (it == v.end()) {
        return 0;
    }
    return *it + mystery(v, it + 1);
}

int main() {
    vector<int> v;
    v.push_back(2);
    v.push_back(4);
    v.push_back(6);
    v.push_back(8);

    cout << mystery(v, v.begin()) << endl;
}

```



- ① begin (2)
- ② 4
- ③ 6
- ④ 8
- ⑤ end

(1) How many times will mystery() be called?

(2) What value will be printed?

(3) What is the running time of mystery() in terms of n ?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n \log n)$
- e) $O(n^2)$

1

$$(n-k) + (n+1-k) + (n+2-k) + \dots$$

average / worst
↓
 $O(n)$

(4) Rewrite the mystery() function using a while loop and iterator instead of recursion. You may use about 7 lines.

```
int mystery_iter(vector<int>& v) {
```

```
}
```



Name:

2. Considering the following code:

```
int mystery(int n) {  
    if (n <= 1) return 1;  
    return mystery(n/2) + mystery(n/4)+1;  
}
```

(1) How many times is mystery() called if we set n=16?

(2) What is the maximum recursion depth?

Hint: Maximum recursion depth is the length of the longest chain of function calls from the initial call to the base case.

(3) Which function runs in $O(n)$? Assuming all operations except the recursive call itself take $O(1)$ time.

- a) $f(n) \{ \text{return } f(n-1)+f(n-1); \}$
- b) $f(n) \{ \text{return } f(n/2)+f(n-2); \}$
- c) $f(n) \{ \text{return } f(n-1)+f(n-2)+f(n-3); \}$
- d) $f(n) \{ \text{return } f(n/2)+f(n/2); \}$

$f(n)/f(n) \rightarrow O(1)$
 $f(n) \rightarrow O(n)$
 $f(n) \rightarrow$

$2^n \cdot \log_2 n$

$2 \log_2 n$

$f(0) = 1$

$f(k) = 2 f(k-1)$

$f(n) = 2^{n-1}$

